Oracle Database @ GCP Automation

Operations Advisory Team
Office of CTO | EMEA Technology Engineering

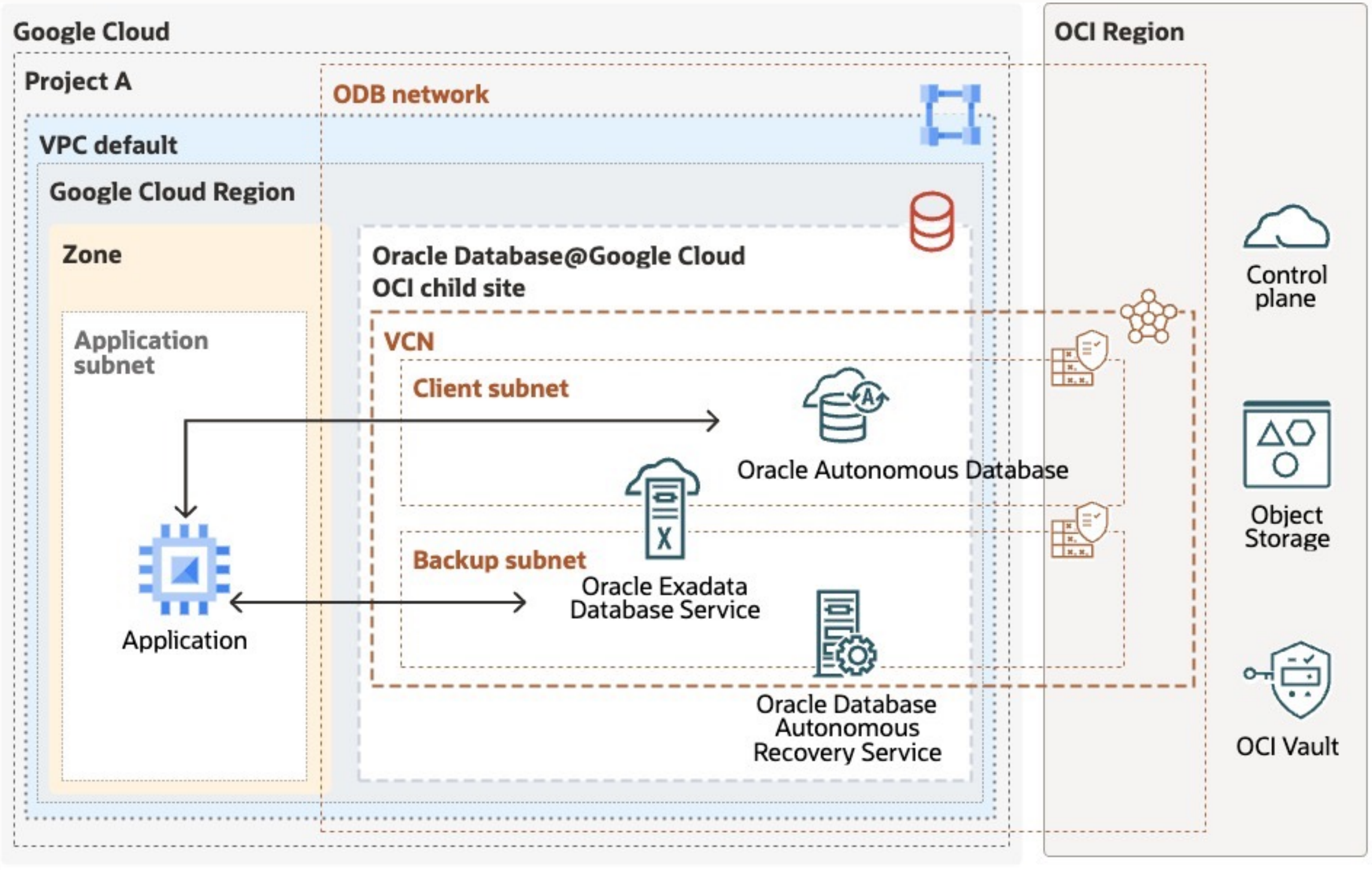
May 2026

Agenda

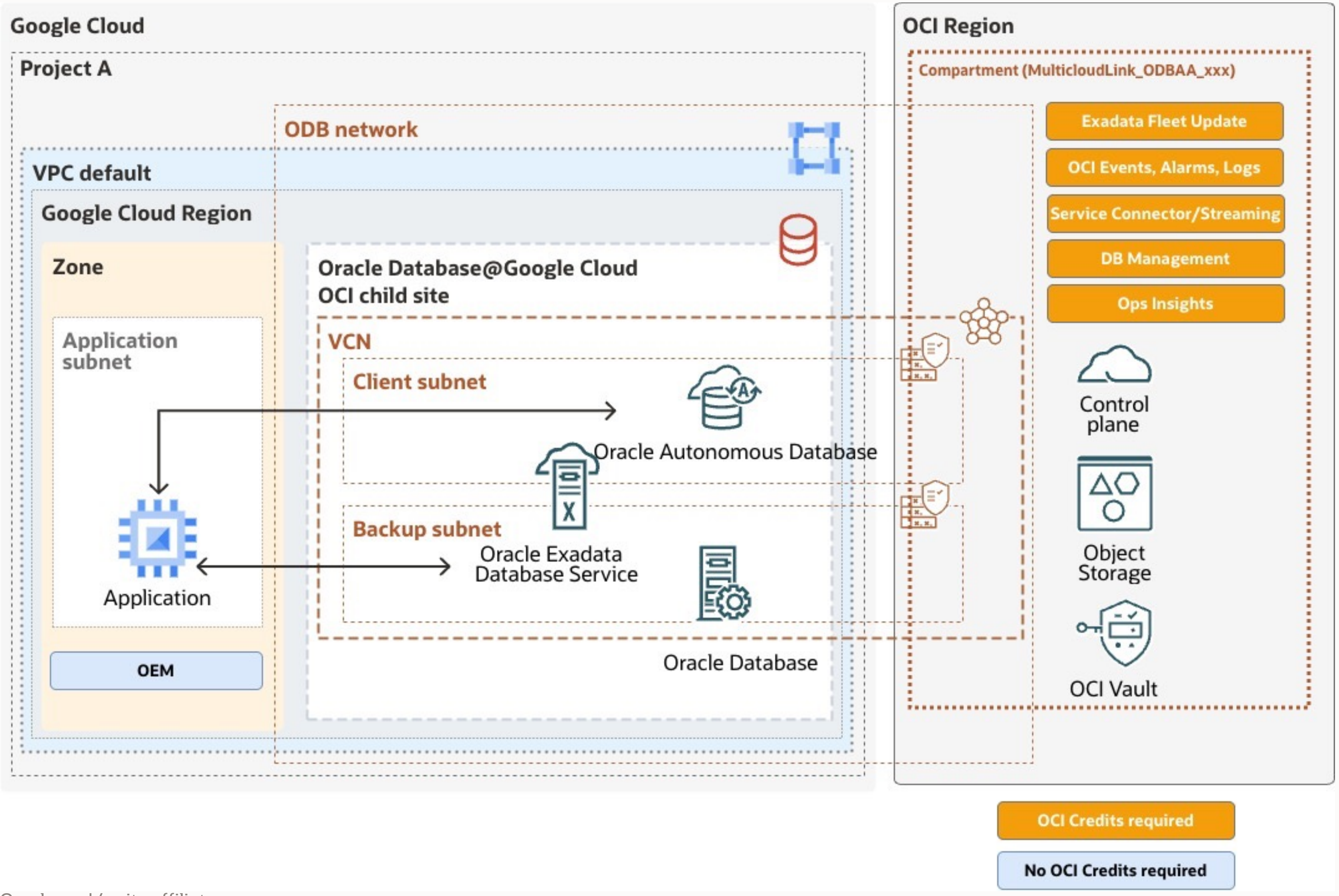
1. Oracle Database at Google Cloud architecture
2. Operations & Interfaces
3. IaC & GitOps

ODB@GCP Architecture

Architecture overview

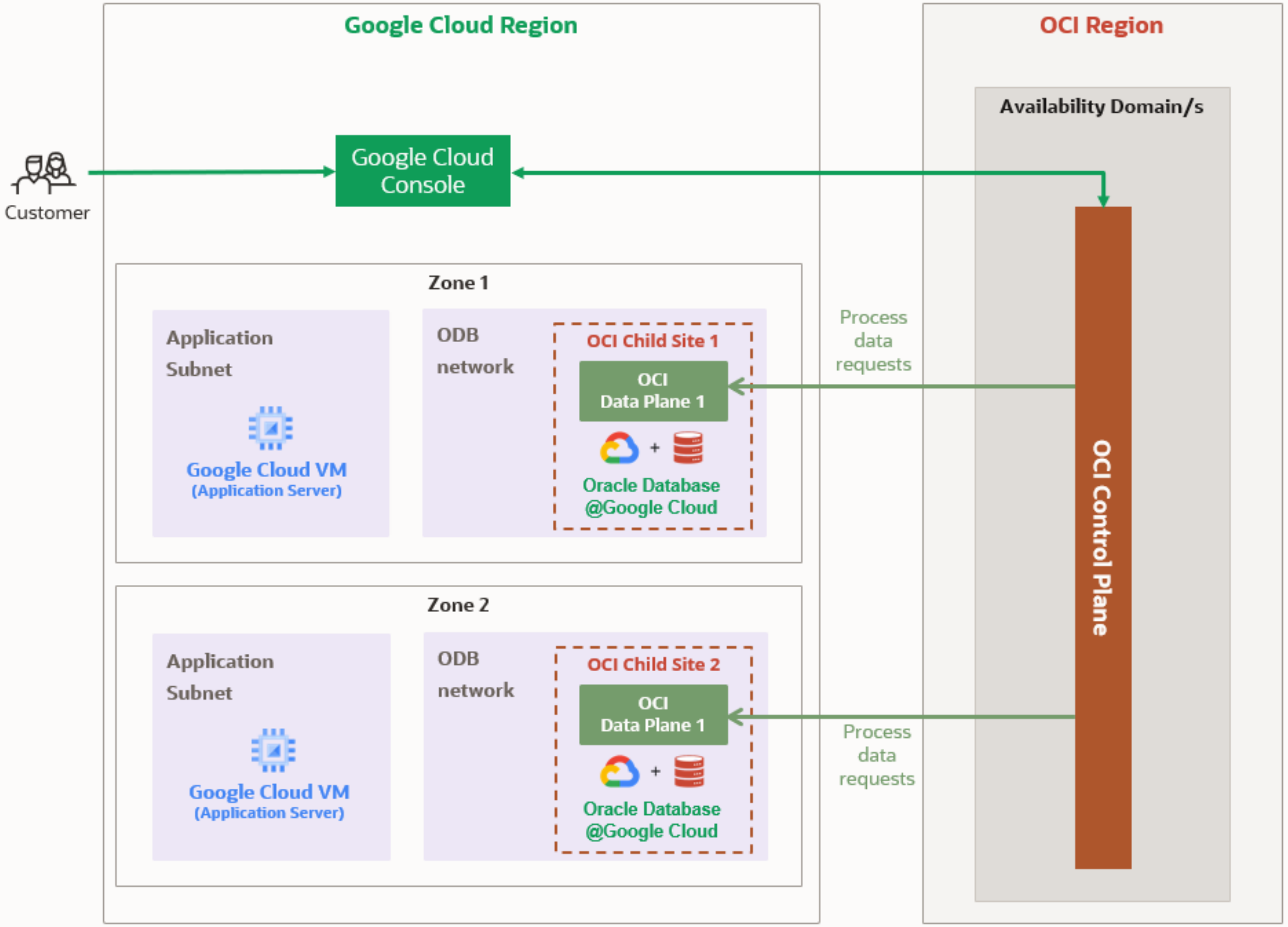


Optional operational services

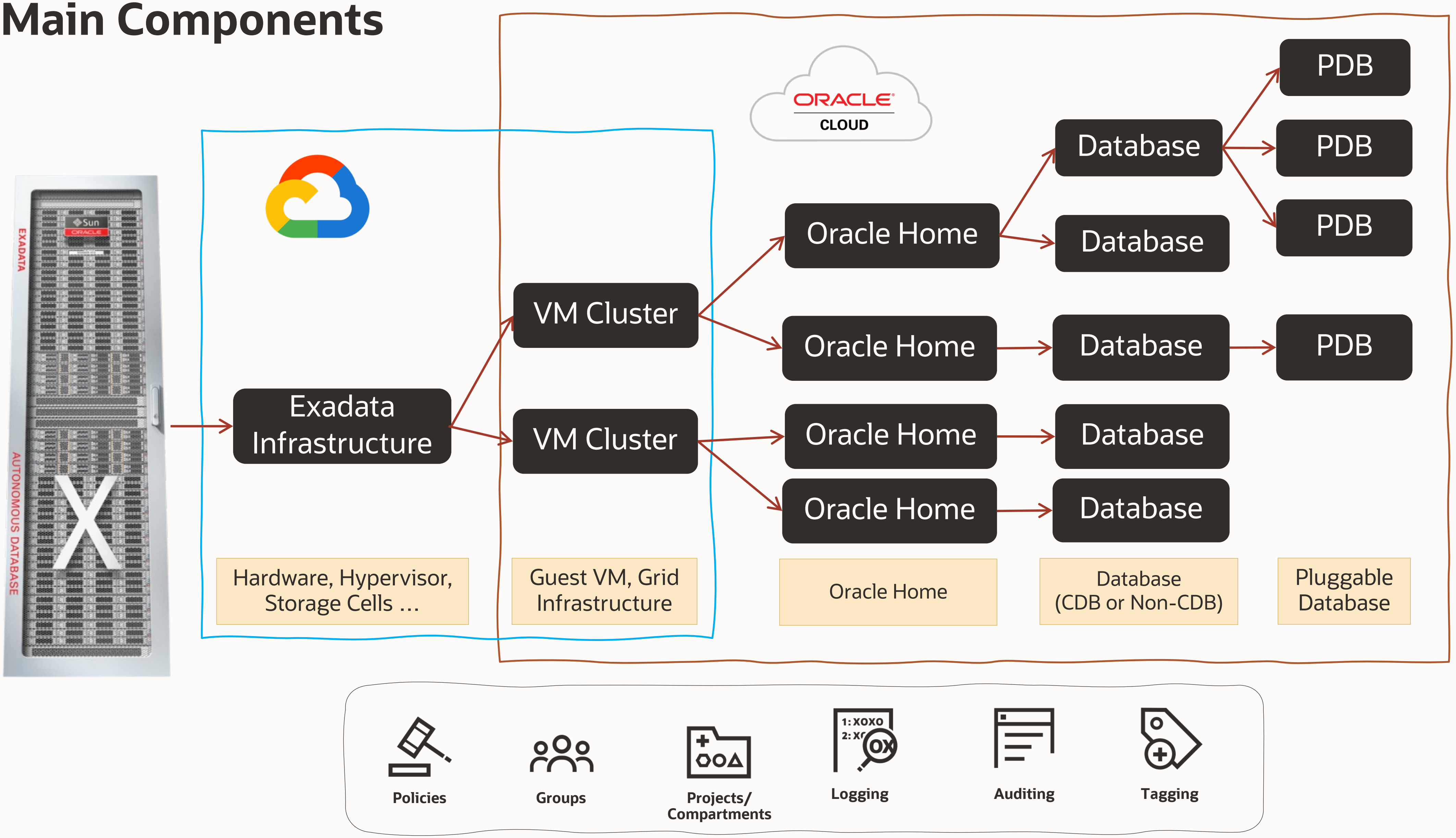


Operations & Interfaces

Control Plane Architecture



Main Components



Management Interfaces

Rest APIs

REST APIs that use HTTPS requests and responses for managing OCI and GCP resources, including Exadata Cloud

Consoles

Intuitive, graphical interfaces for creating and managing your OCI and GCP resources, are one of the interfaces to the Rest APIs

Command Line Interfaces (OCI CLI / gcloud CLI)

Lightweight tools for command-line tasks. Support scripts and extend Console functionality

Software Development Kits (SDKs)

Enable custom solutions for Exadata Cloud and OCI services

DevOps Tools & Plug-ins

Tools like Terraform Provider and Ansible Collection for provisioning and automation

Cloud Shell

Browser-based terminal with pre-authenticated CLI, useful for tutorials and quick commands

Local VM Command-Line Interfaces

CLI utilities located on the VM guests

Provisioned as part of the VM clusters on the Exadata Cloud Infrastructure, are available to perform various lifecycle and administration operations

dbaascli

Use the dbaascli utility to perform various database lifecycle and administration operations on the Exadata Cloud Infrastructure such as changing the password of a database user, starting a database, managing pluggable databases (PDBs), scaling the CPU core count in disconnected mode, and more.

ExaCli

Use the ExaCLI command-line utility to perform monitoring and management functions on Exadata storage servers in the Exadata Cloud.

bkup_api

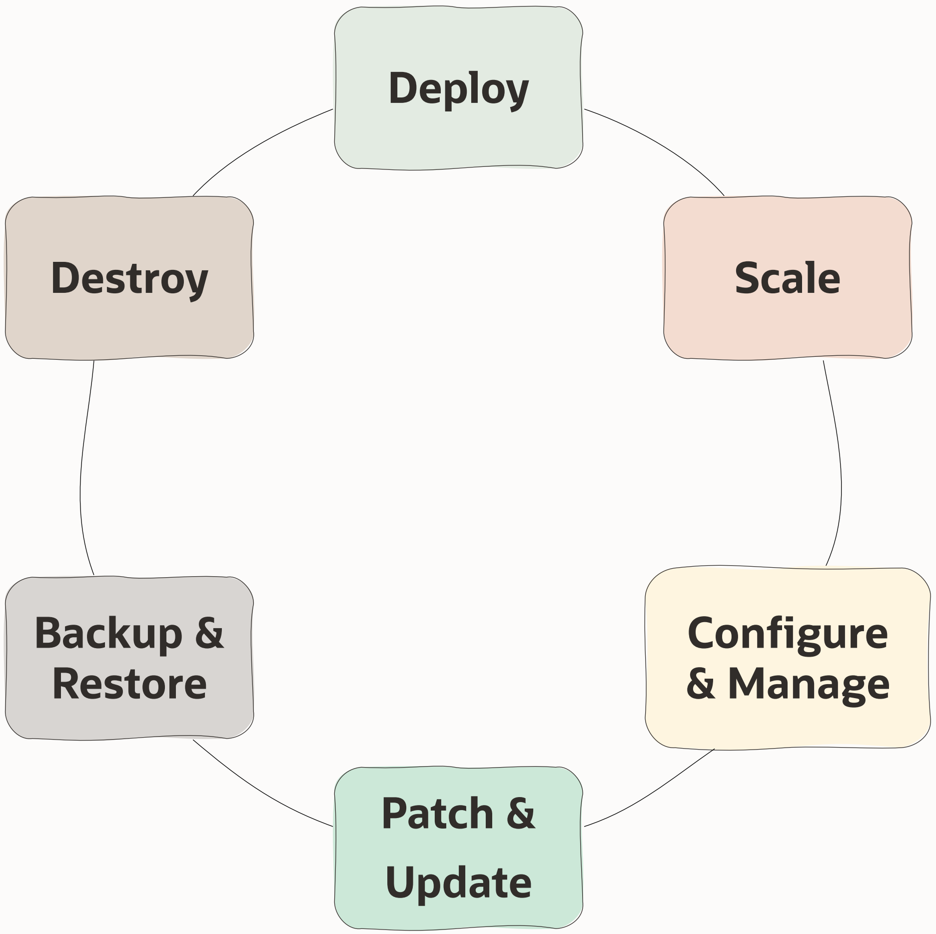
Handles backup/recovery tasks, including on-demand database and PDB backups.

DEPRECATED

Must be connected to a VM in the Exadata Cloud VM cluster; uses VM OS user security, not OCI user security.

Changes sync to OCI Control Plane via ***DB Sync***.
Tools are auto-updated by Oracle on a regular basis.

Operations



Deploy
Infrastructure, VM Clusters, Oracle Home, Database…

Scale
CPU, Memory, Local Storage, ASM Storage…

Configure & Manage
Set parameters, Create users, Start/Stop,…

Patch/Update
Infrastructure, OS, Grid Infrastructure, Database…

Backup
Automatic Backups, OnDemand Backups, Database tools (dbaascli, RMAN)…

Destroy
Infrastructure, VM Clusters, Oracle Home, Database…

Where I do what?

- ODB Network and subnets
  - Creation/Deletion
  - View information
- Exadata infrastructure
  - Creation/deletion
  - View Information
- Exadata VM clusters
  - VM cluster creation/deletion
  - View information

- *Oracle CDBs and PDBs aren't provisioned and managed on Google Cloud but in OCI*

ORACLE CLOUD

- Exadata infrastructure
  - Configuration changes
  - Updates/Upgrades planning
- Exadata VM clusters
  - Status management
  - Configuration changes
  - Updates/Upgrades
  - GI management
  - Scale OCPU/Memory/Storage
- Oracle Home creation/deletion
- Oracle container databases and pluggable databases
  - Creation/Deletion
  - Status management
  - Updates/Upgrades
  - Backup & Recovery
  - DR setup and management
  - (…)

Management Interfaces vs. Operations

Operation	Google cloud				OCI					IaC	
	Console	gcloud	Rest API	SDK	Console	OCI CLI	dbaascli	Rest API	SDK	Terraform[1]	Ansible Collections
Create/Delete ODB Network and ODB Subnets	✓	✓	✓	✓	✗	✗	✗	✗	✗	✓	✗
Create/Delete Exadata Infrastructure instances	✓	✓	✓	✓	✗	✗	✗	✗	✗	✓	✗
Scale Exadata Infrastructure instances	✗	✗	✗	✗	✓	✓	✗	✓	✓	✗	✗
Create/Delete Exadata VM Cluster	✓	✓	✓	✓	✗	✗	✗	✗	✗	✓	✗
Stop/Start/Restart VM clusters	✗	✗	✗	✗	✓	✓	✗	✓	✓	✗	✓
Create/Delete Database Oracle Homes	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓
Create/Delete Container Databases	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓
Stop/Start/Restart Container Databases	✗	✗	✗	✗	✓	✓	✓	✓	✓	✗	✓
Create/Delete/Clon Pluggable Databases	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓
Stop/Start/Restart Pluggable Databases	✗	✗	✗	✗	✓	✓	✓	✓	✓	✗	✓
Patch Guest OS/GI	✗	✗	✗	✗	✓	✓	✗	✓	✓	✗	✓
Patch CDB/PDB	✗	✗	✗	✗	✓	✓	✓	✓	✓	✗	✓
Database Backup&Restore	✗	✗	✗	✗	✓	✓	✓	✓	✓	✗	✓
Scale VM OCPU/Memory/Storage (Local/Shared)	✗	✗	✗	✗	✓	✓	✗	✓	✓	✗	✓
Schedule Infrastructure Maintenance	✗	✗	✗	✗	✓	✓	✗	✓	✓	✓	✗
Monthly Security Maintenance	✗	✗	✗	✗	✓	✓	✗	✓	✓	✗	✗
Create software images	✗	✗	✗	✗	✓	✓	✗	✓	✓	✓	✓
Create/Delete Data Guard Association	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓
Perform Switchover/Failover	✗	✗	✗	✗	✓	✓	✓	✓	✓	✗	✓

[1] Use the appropriate Terraform provider depending on the component

IaC & GitOps

What is and does Terraform

Terraform is a **declarative** language that defines the **desired state** of infrastructure.

It uses a **state file** to consistently provision **Landing Zones (LZ)** and **workload** resources across **any cloud.**

Terraform is great to **provision the Landing Zone and workload** resources.

Declarative Language

Defines the "**what**", not the "how". A simple configuration expresses the entire desired state.

State Management

A **state file** acts as the **single source of truth**, mapping configurations to real-world resources.

Landing Zones Workload Resources

LZ set the **foundation** and **guardrails**. **Workload resources** add the stacks on top across **any cloud**.

OCI Terraform Provider Resources

Terraform | Google Cloud

Exadata Infrastructure
cloud_exadata_infrastructure

VM Cluster
cloud_vm_cluster

Autonomous Database
autonomous_database

Terraform | ORACLE CLOUD

DB Home
oci_database_db_home

CDB
oci_database_database

PDB
oci_database_pluggable_database

Terraform is not enough

The Limits

Terraform has some boundaries. It cannot manage tasks inside a resource, only the resource itself.

- **No patching:** Not for OS/DB/app patching or upgrades
- **No in-guest config**: Doesn't install packages, edit files, or manage services on existing hosts
- **No app/data config**: doesn't manage DB schemas/users or app settings.
- **Non-Updatable** changes (needs to re-deploy)
- **Last resort only**: Provisioners (remote-exec/local-exec) and generic helpers (terraform_data, deprecated null_resource) aren't suited for Config Management and can leak secrets to state/logs

The Solution

Use the lifecycle statement block to **bridge the gap** between **declarative IaC and procedural** realities.

```
lifecycle {
      ignore_changes = [
        database[0].admin_password,             <--- Ignoring by default
        database[0].database_software_image_id] <--- ignore db_home patching
        database_software_image_id,             <--- ignore db_home patching
        db_version,                             <--- ignore db_home patching
   }
```

The ignore_changes skip diffs for specific fields (config edits or drift)

What is Ansible

Ansible is a declarative, **YAML-based** automation engine, **agentless** IaC tool used primarily for **configuration management**.

Agent less

No permanent **agents** on managed systems.
Uses **temporary** network **connections** via **SSH** or Windows Remote Management

Vs. Alternatives

Chef, Puppet and Salt require installed agents.
Ansible eliminates this **complexity** and overhead

Minimum Requirement

Only requires Python installed on managed systems.
Nothing more

Central Orchestration

Centralized and efficient **architecture**
All **orchestration** is done from a **control node** (masterd host).

SSH Security

Leverages native **SSH security** without additional ports or running daemons

Zero Overhead

Zero resource consumption on managed servers when no tasks are running

Playbooks: The Heart of Ansible

Playbooks are the language by which Ansible orchestrates, configures, administers, or deploys systems.

Execution flow:

Playbooks → Roles → Tasks → Hosts

GIT Versioning

Playbooks can be stored and versioned in GIT (SSoT), allowing full control of code

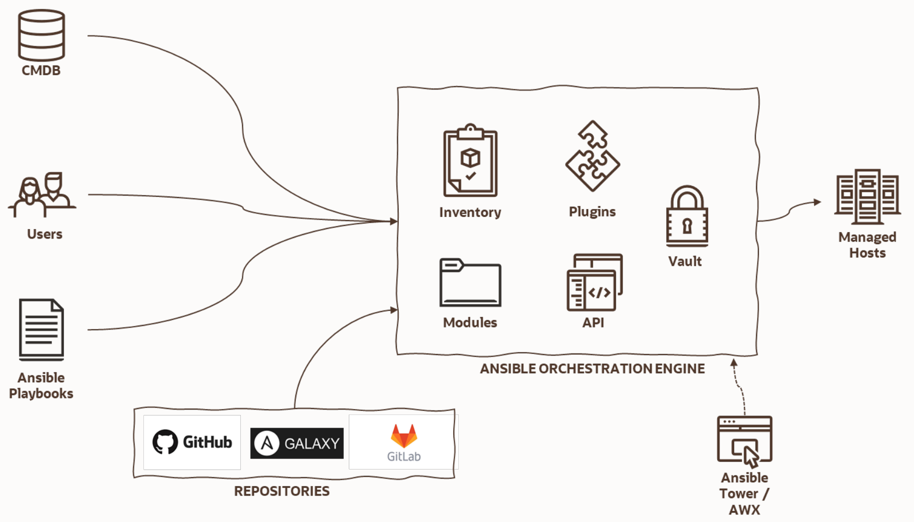
Code Reusability

Import playbooks (collections) for shared common code across teams and projects

CI/CD Integration

TERRAFORM | JENKINS | GITLAB | OTHER

Ansible can be used as an executor in CI/CD runners, integrating seamlessly with tools like Terraform, Jenkins and more



OCI Ansible collection

Ansible Colletion helps you **automate the provisioning and configuring** of Oracle Cloud Infrastructure services and resources

Github repository

OCI Ansible collection

Ansible Tower/ AWX Integration

Ansible Tower

AWX.

Ansibile Inventory Plugin

Static Inventory & Dynamic inventory

Supported Services

The Oracle Cloud Infrastructure Ansible Collection provides **hundreds of modules** to **automate** provisioning and configuration of **OCI resources**.

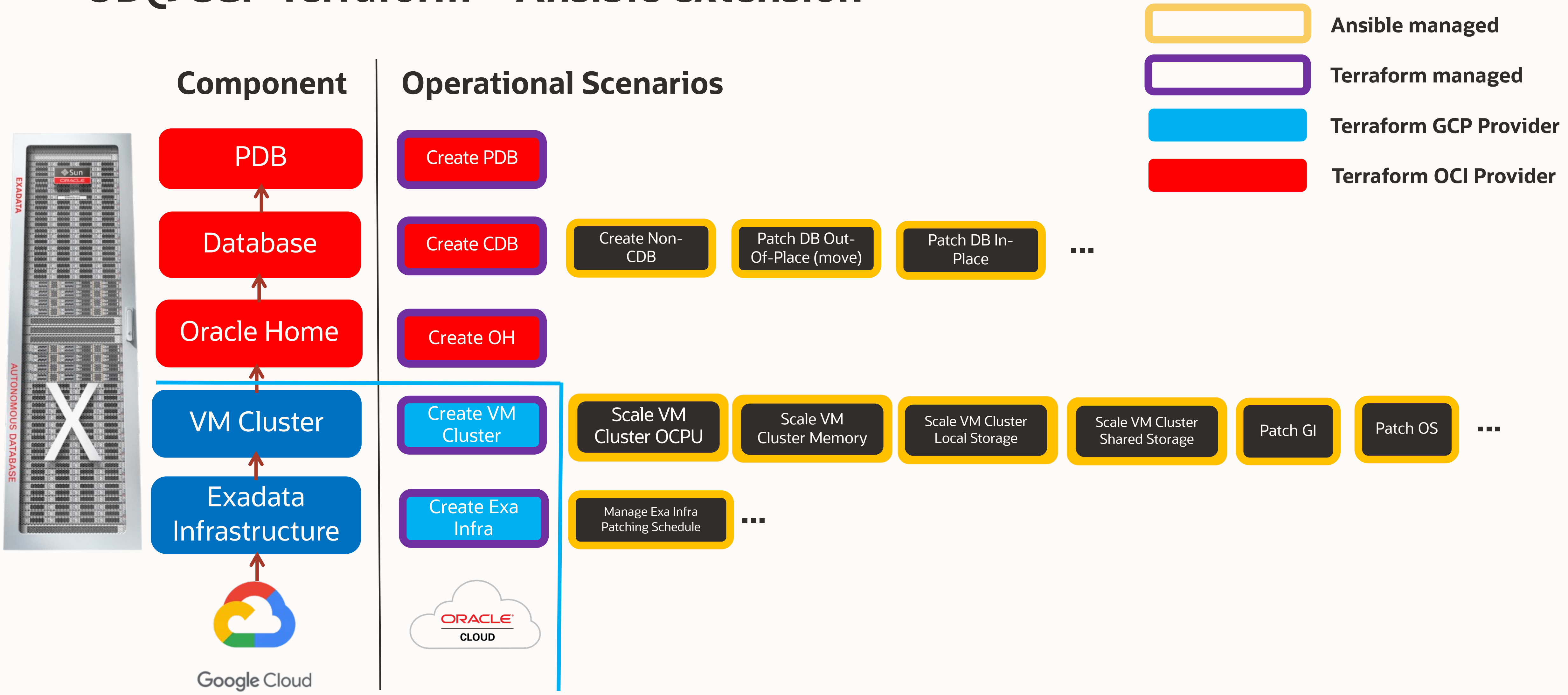
Minimal Requirements

The OCI **Ansible collection** requires Ansible version. **2.9 or later**

OCI Collection key benefits

Provision & configure OCI via Ansible.
Dynamic inventory for discovery.
Centralized runs (AWX/Tower, OLAM).
Broad, Oracle-maintained coverage.

OD@CGP Terraform + Ansible extension



Wrap up & Next Steps

Takeaways | Next steps

- There is ***not a single tool to manage everything***.
- Different management interfaces to manage ExaDB-D components, ***leverage to your preferred automation tooling***.
- The use of a ***GitOps*** approach ***simplifies your operations***.
- ***Exadata Fleet Update facilitates*** the schedule, patching process and automation needed.
- OCI Basic monitoring & integration services can be used to forward valuable information to your IT Systems (SIEM, ITSM, etc.).

Additional Operations Workshops

Exadata Fleet Update

Learn what is Exadata Fleet Update and how it can be used to patch Exadata OS, GI & DBs at scale

OD@ExaDB-D Operations Deep Dive

Learn how you can use different management interfaces to provision, change, maintain & manage high availability for Exadata Cloud Service in GCP

OD@ExaDB-D Operations with Ansible Deep Dive

Learn how you can use Ansible to provision, change, maintain & manage high availability for Exadata Cloud Service in GCP

Stay tuned

[Operations Advisory GitHub Repository]

Thank you

ORACLE